VaultX Cyber Tech AI Internship

Week 6 Report

AI Business Automation: Vehicle Service Center
Smart Booking Bot (WhatsApp Delivery)

Intern	Aziz ur Rehman
Program	BS Artificial Intelligence
Internship	VaultX Cyber Tech AI
Week	Week 6 (AI Business Automation)
Repository	chaziz-ai/vaultx-ai-internship (branch: main)

1. Project Overview

The Week 6 assignment asked for a practical AI-powered automation solution for a real SME or local business in Pakistan, following the path **Real Problem** → **AI Solution** → **Automation** → **Business Value**. This report follows that path.

Project

Vehicle Service Center: Smart Booking + Parts-Aware Scheduler. A WhatsApp bot that books bike and car services, checks the calendar and the parts inventory before confirming, and keeps the stock sheet up to date.

Real Problem

A service center that takes bookings by hand over WhatsApp faces double bookings, bookings on closed days, bookings for services whose parts are out of stock, and no automatic warning when stock runs low.

AI Solution

An AI agent chats with the customer in English or Roman Urdu, collects the service, vehicle type and preferred time, and uses three tools: calendar availability, inventory check, and event creation. The event is created only after the customer explicitly confirms.

Automation

- Calendar event created automatically after confirmation.
- Parts used by the service are deducted from stock and logged in a Booking Log sheet.
- A second workflow checks stock twice a day and sends the owner a low-stock WhatsApp alert.
- A Friday safety net automatically cancels a booking if one is wrongly made on the closed day.

Business Value

- Instant replies and bookings without staff involvement.
- No bookings on closed days or into occupied slots.
- Stock is tracked automatically and the owner is warned before parts run out.
- A Booking Log records which parts each service consumed.

Business Rules

Hours 9am to 6pm, Friday closed, 2 parallel service bays. Service durations (minutes, bike / car): Oil Change 15/20, Washing 20/30, Tyre Puncture or Tube Change 10/15, General Service 40/50, Battery Check or Replace 15/20, AC Service or Gas Refill (car only) 30.

Why this matters

Week 5 ended with WhatsApp delivery blocked by Meta webhook issues. This week WhatsApp delivery works through GreenAPI, and the tool-calling idea from Week 5 is applied to a real business process instead of a general question-answering agent.

2. Tech Stack & Architecture

Tools and Technologies Used

- **n8n** (self-hosted, local) — builds both workflows; carried over from Weeks 3 and 5.
- **OpenAI gpt-4o-mini** with **Simple Memory** — the AI Agent's language model and per-customer conversation memory.
- **GreenAPI** — sends and receives WhatsApp messages.
- **ngrok** — exposes the local n8n webhook to the internet so GreenAPI can reach it.
- **Google Calendar** — holds the bookings. **Google Sheets** — inventory (Sheet1) and Booking Log.

Data Flow (Booking Agent)

```
WhatsApp → GreenAPI → ngrok → n8n Webhook
→ Edit Fields (30-day date table + current time, computed by code)
→ AI Agent (gpt-4o-mini + Simple Memory; tools: Calendar get events, Sheets get
stock, Calendar create event)
→ Get many events → Code (customer's booking, part used, Friday check)
→ If1 (Friday violation?) → Delete event → fixed cancellation message
→ If (part used?) → Split Out → Get stock → Calculate quantity → Update stock row
→ Append Booking Log row
→ HTTP Request (GreenAPI sendMessage) → customer on WhatsApp
```

Data Flow (Low Stock Alert)

```
Schedule Trigger (9am and 4pm) → Get all parts → Filter (stock ≤ threshold) → Code
(one combined message) → HTTP Request (WhatsApp to the owner)
```

Design Principle: Code for Facts, AI for Conversation

Early tests showed that the model made mistakes on dates, weekdays and opening hours even with clear instructions. These are fixed, checkable facts, so they are now calculated by n8n expressions and handed to the agent, which only reads them. The AI is used for the conversation and for deciding which tool to call.

3. Implementation Summary

Components Built

- **booking-agent-workflow.json** — the main WhatsApp booking workflow.
- **low-stock-alert-workflow.json** — the scheduled stock alert.

- Google Calendar "Vehicle Service Center Bookings" and Google Sheet with Sheet1 (inventory) and Booking Log tabs.
- A system prompt with critical rules: language matching, date-table lookup, inventory check, part-name mapping, business hours, parts consumption mapping.

Node	Role
Webhook	Receives the customer's WhatsApp message from GreenAPI.
Edit Fields	Adds dateContext (next 30 days with weekday, Friday tagged CLOSED) and currentTime.
AI Agent	Talks to the customer and chooses tools; booking only after confirmation.
Simple Memory	Remembers each customer's conversation (session = WhatsApp chatId).
Get many events + Code	Finds this customer's latest booking, reads the part used, checks for a Friday booking.
If1 + Delete event	Cancels a booking that was wrongly made on a Friday.
If + Split Out	Runs the stock update only when a part is used; splits several parts into separate items.
Update row / Append row	Reduces stock in Sheet1; writes a row to Booking Log.
HTTP Request	Sends the reply to the customer through GreenAPI.

Service to Part Mapping (parts consumption)

Oil Change uses Engine Oil and Oil Filter; Tyre Puncture uses Patch Kit; Tube Change uses Tube; Battery Replace uses Battery; AC Service uses AC Filter; AC Gas Refill uses AC Gas. Washing, General Service and Battery Check use no part.

Reused from Previous Weeks

n8n workflow building and the error-handling habits from Week 3, the webhook and ngrok setup and the "code over LLM" lesson on dates from Week 5, and the tool-calling agent pattern from Week 5, now with Calendar and Sheets as the tools.

4. Testing & Evidence

4.1 Webhook Tunnel Verified

ngrok forwards the public address to n8n on port 5678. The request log shows GreenAPI's POST requests reaching the n8n webhook path with status 200.

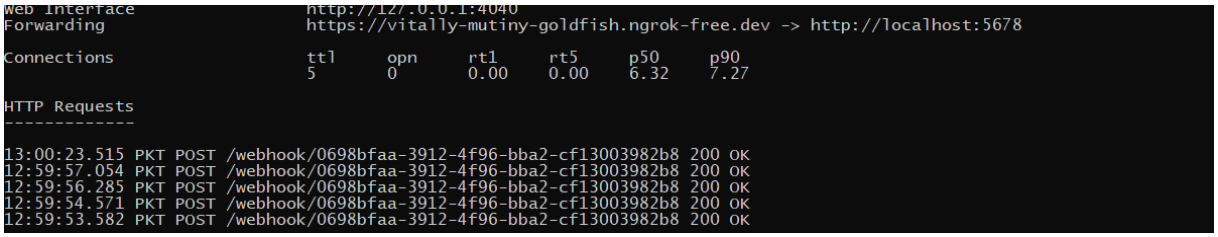


Fig 1: ngrok forwarding to localhost:5678 and incoming POST requests to the n8n webhook, all returning 200 OK.

4.2 Live Booking Conversation on WhatsApp

The bot was tested through real WhatsApp messages. In the conversation below the customer books a car tube change, books a patch kit, cancels bookings in natural language, and finally asks about future bookings; the agent answers correctly each time.

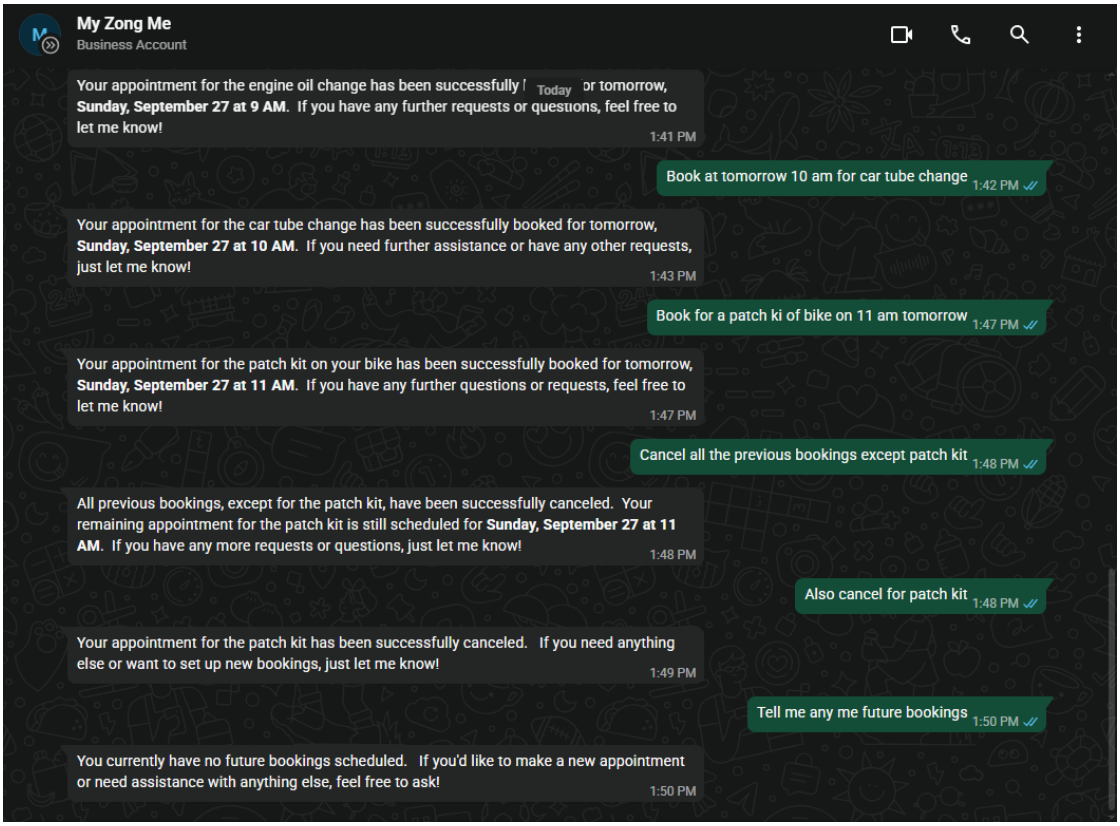


Fig 2: WhatsApp conversation showing booking, cancellation and a bookings query handled by the agent.

4.3 Stock Tracking and Booking Log

After a confirmed booking the matching part is reduced in Sheet1 and a row is appended to the Booking Log with the date, service, part used, quantity and customer number.

	A	B	C	D	E
1	Date	Service	Part Used	Quantity Used	Customer Number
2	26 Sep 2026, 13:54	Tube Change - Bike - 923276374572@c.us	Tube	1	923276374572@c.us
3	26 Sep 2026, 13:59	Battery Change - Bike - 923276374572@c.us	Battery	1	923276374572@c.us

Fig 3: Booking Log rows written automatically after bookings (Tube and Battery).

4.4 Complete Workflow

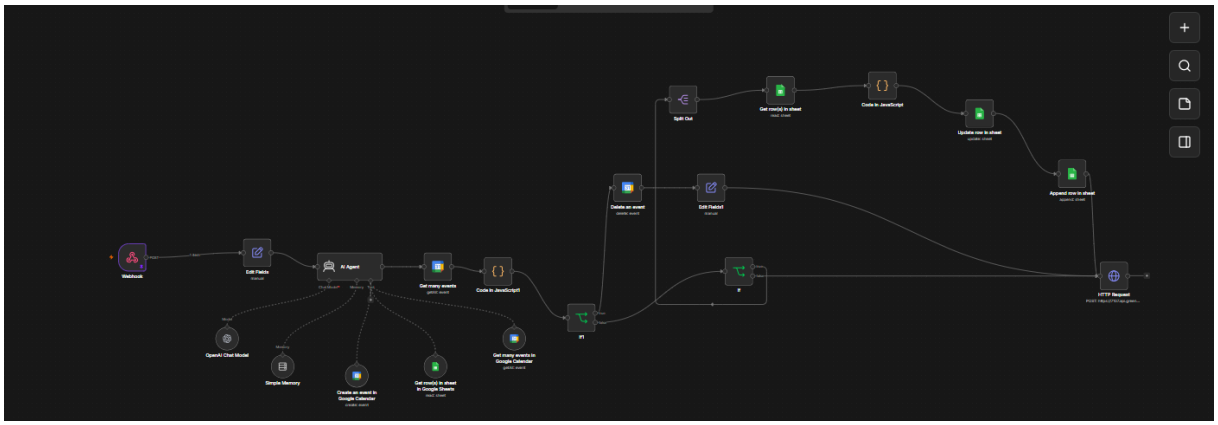


Fig 4: The complete Week 6 Project workflow in n8n, from Webhook to the final HTTP Request.

4.5 Test Summary

Test	Result
Booking flow (wash, tube change, patch kit, battery)	Pass: Calendar event created and reply received
Inventory check (e.g. tyre tube availability)	Pass after fixes in Section 5
Friday-closed rule	Pass: refused and suggested another day
Relative dates (kal, parso, 3 din baad)	Pass after moving date logic to code
Business hours (is the shop open now)	Pass after adding currentTime
Unrelated messages (Hi, Who are you)	Pass: clean explanation of the bot's role
Parts auto-decrement and Booking Log	Pass: stock reduced, log row written, Part Used column fixed
Low Stock Alert workflow	Pass: real WhatsApp alert received
Language matching (English vs Roman Urdu)	Not fully passing; see Section 6

5. Refinements After Initial Testing

Live testing exposed several issues. Each one is recorded below with its cause and fix.

5.1 Dates and Weekdays Were Unreliable

The agent miscalculated relative dates (for example "3 din baad") and paired the wrong weekday with a date, even when the correct date table was in its input. **Fix:** the dates are built by an Edit Fields node using n8n code, including a [CLOSED - FRIDAY] tag, and the prompt tells the agent to copy a row exactly. A code check after the agent also detects a Friday booking and cancels it automatically, because prompt rules alone were not fully reliable.

5.2 No Reply on Simple Messages

After a greeting, the WhatsApp reply never arrived even though the AI Agent had produced an answer. The "Get many events" node found no booking, returned zero items, and n8n stops a branch when a node outputs nothing, so the final HTTP Request never ran. **Fix:** turn on "Always Output Data" for that node.

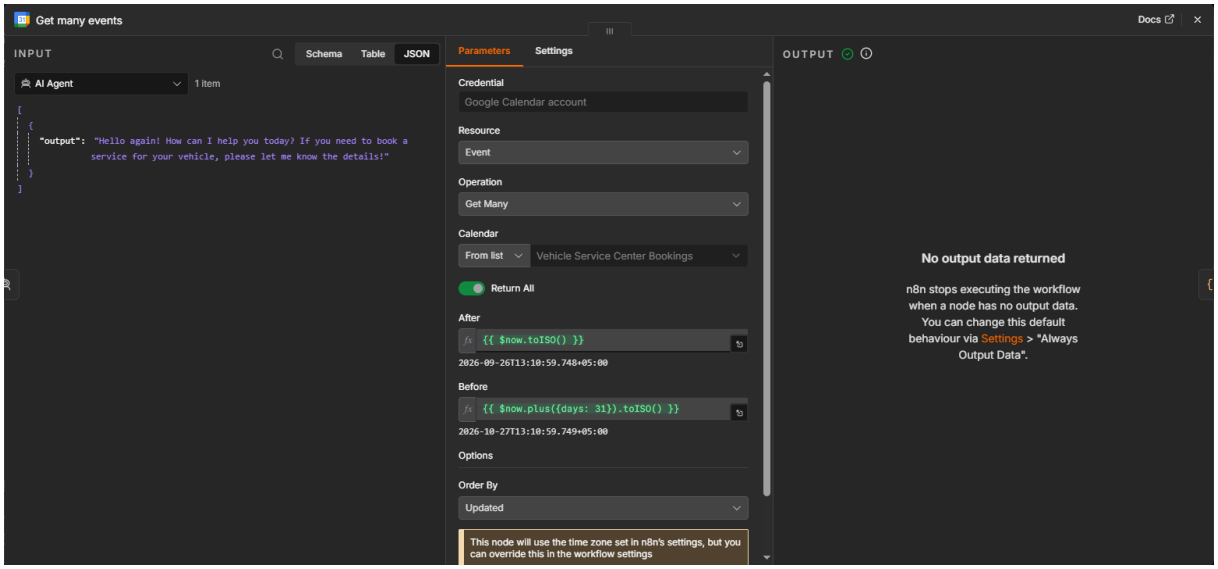


Fig 5: "No output data returned" on the Get many events node, which stopped the chain before the reply was sent.

5.3 Part Used Column Was Empty

The Booking Log "Part Used" cell stayed empty. The field read `$json.partName`, which does not exist, and `$json` only refers to the previous node. **Fix:** read the value by node name, `$('Split Out').item.json.partsArray`, after which the log showed the correct part.

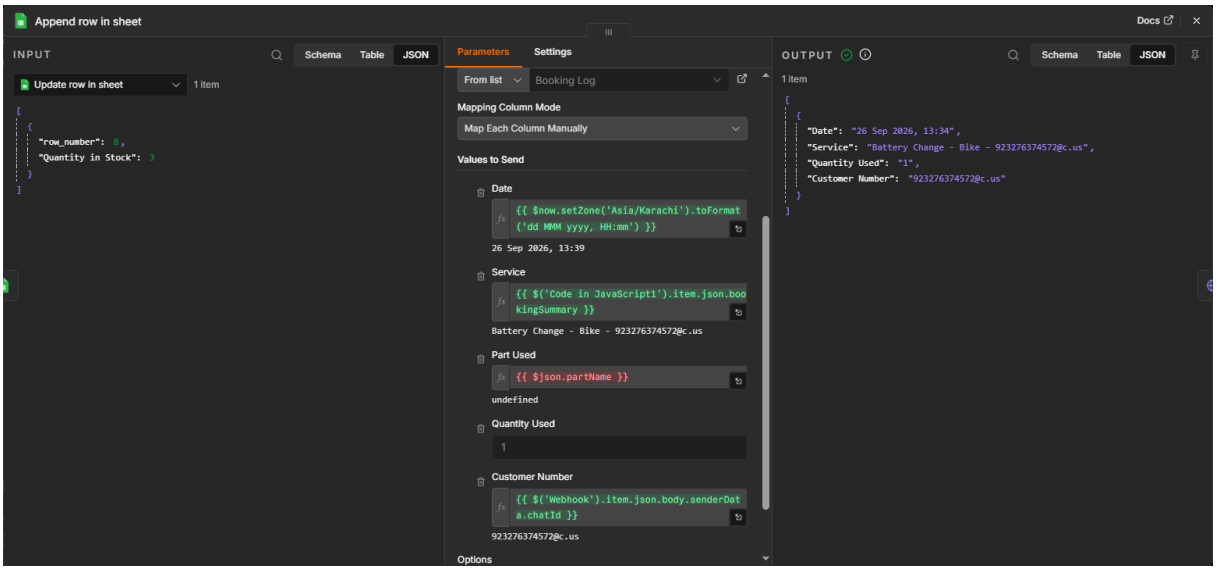


Fig 6: The Part Used field evaluating to undefined before the fix.

5.4 Inventory, Opening Hours and Names

- The agent sometimes skipped the stock check, and "tyre tube" did not match the sheet name "Tube". Fixed with a critical inventory rule and an exact part-name mapping in the prompt.
- The Sheets node returned 404 because the tab is named Sheet1, not ID 0. Fixed by selecting the document and sheet from the list.
- The shop was reported open at night because the agent had dates but no time. Fixed by adding currentTime and a business-hours rule.
- Pasting literal double curly braces into the prompt broke the agent, because n8n evaluates them as an expression. The braces were removed.
- The reply request broke on newlines and quotes in the AI text. The HTTP body now uses "Using Fields Below", which escapes them.
- The agent asked for the customer's name because the event title description said "CustomerName". The description was changed and the phone number is appended by an expression instead.

5.5 Secret Handling

The GreenAPI token appeared inside an exported workflow JSON. Before committing, the token was revoked in GreenAPI, a new one was issued and set in n8n, and the files in the repository contain the placeholder YOUR_API_TOKEN. Credentials are never committed.

6. Known Limitations

These issues were found and are documented rather than hidden:

- **Language matching:** the bot sometimes replies in Roman Urdu to an English message, despite three rounds of prompt changes. A clean retest is still pending.
- **Cancellation and stock:** cancelling a booking still reduces part stock. It should not, or should restore it.
- **Booking Log date:** the Date column records when the row was written, not the date and time of the booking.
- **Possible duplicate reply:** when a service uses two parts, Split Out creates two items and the final HTTP Request ran twice (Fig 7). The planned fix is an Aggregate node before it; this was not applied at the time of writing.
- **Local setup:** n8n and ngrok run on a local machine, and the free ngrok URL changes after a restart, so the GreenAPI webhook URL must be updated each time. Conversation memory is kept in n8n itself, not in a database.

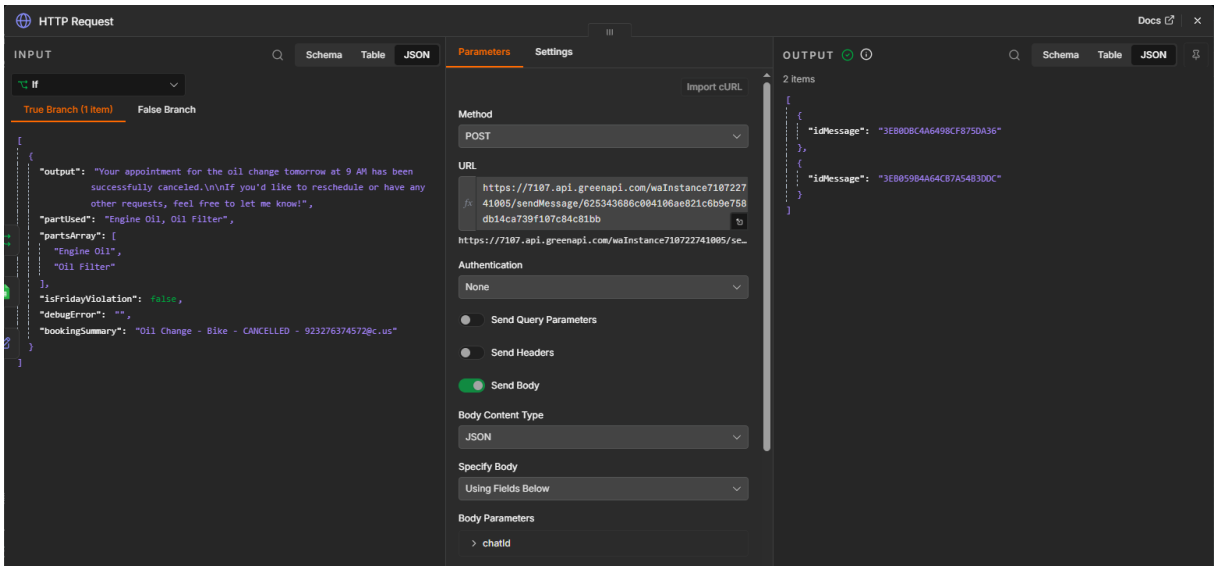


Fig 7: HTTP Request output showing 2 items (two idMessage values), i.e. two WhatsApp messages sent for one booking.

7. Conclusion / Learnings

This week turned the Week 5 tool-calling agent into a working business automation: a WhatsApp bot that books, checks stock, updates the inventory and alerts the owner, tested through live WhatsApp messages. The clearest lesson was that an LLM should not be trusted with facts that code can calculate exactly. Moving dates, weekdays and hours out of the model removed most of the errors, and a code-based safety net caught the rest.

Three n8n behaviours cost the most debugging time: a node with zero output stops its branch, `$json` only refers to the previous node, and every node after a Split Out runs once per item. Knowing these will speed up future workflows.

Finally, the unresolved items in Section 6 are listed on purpose. A working system with honest, documented limits is more useful than one that claims everything is finished.